

MINI-PROJECT SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)
Mini-Project Title: Mini-Project 3: OCR Expense Tracker & Receipt Parser
Team / Student Name: Le Huu Thai (Mobile Engineering Lab)
Submission Date: 08/10/2026

1. GENERAL INFORMATION & DELIVERABLE LINKS

- Team Members:**
 - Le Huu Thai — Student ID: 23IT.EB091 — Role: Lead Architecture & On-Device ML/OCR Engine / CustomPainter Visualization, UI & SQLite Storage — Contribution: 100%
-  **Live Demo / Web App:** <https://ocr-expense-tracker-phi.vercel.app>
(Native Android Debug APK: `build/app/outputs/flutter-apk/app-debug.apk`, 192 MB)
-  **GitHub Repository:** <https://github.com/HuuThai127/ocr-expense-tracker>
-  **Video Demo Walkthrough:** [Google Drive Walkthrough Video](#)

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details & Acceptance Level
1	Camera Preview & Framing Overlay	✓ Complete	Native Android/iOS uses the Flutter camera plugin using native camera APIs with hardware flash modes (torch/auto/off) and interactive tap-to-focus via <code>setFocusPoint()</code> . Web uses an inline browser camera preview with the same receipt framing overlay; tap-to-focus and hardware torch are gracefully disabled due to browser API limitations.
2	Receipt Capture & Crop	✓ Complete	Photo capture via <code>takePicture()</code> . Interactive draggable crop box using <code>image: ^4.10.1</code> (<code>copyCrop</code> , <code>encodeJpg</code>) and skip crop option.
3	Google ML Kit On-Device OCR	✓ Complete	100% on-device neural text recognition via <code>google_mlkit_text_recognition: ^0.16.0</code> . Zero cloud APIs. Stopwatch latency tracking measured at 1,241 ms for native camera frame and 2,194 ms for sample bitmap in test environment (exceeding sub-100ms ideal target).
4	ReceiptParser Heuristics	✓ Complete	Multi-pass regex extracting merchant name, Vietnamese currency with dot/comma thousand separators (<code>150.000 đ</code> , <code>150,000 VND</code>), and transaction dates.
5	Review & Edit Screen	✓ Complete	Review screen allowing manual editing of merchant, amount, calendar date picker, category selector chips, and bottom sheet raw OCR text inspector with measured inference latency.
6	SQLite Dual Persistence	✓ Complete	Native Android/iOS uses <code>sqflite: ^2.4.2+1</code> ; Web uses <code>sqflite_common_ffi_web: ^1.1.1</code> (SQLite WASM + IndexedDB). Persistent schema with indexed transaction date surviving restarts and reloads.
7	CustomPainter Donut Chart	✓ Complete	Pure canvas rendering (<code>CategoryDonutPainter</code>) with ease-out cubic animation and touch-activated segment pop-out and center hole details.
8	CustomPainter Bar Chart	✓ Complete	Pure canvas rendering (<code>WeeklyBarPainter</code>) showing Monday–Sunday distribution with animated height and touch hit testing.
9	Web Compatibility & Demo	✓ Complete	Production Flutter Web release deployed to Vercel with SPA routing rewrite (<code>vercel.json</code>), responsive layout, and built-in sample receipt demo flow.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

The application uses an **MVVM-style Layered Architecture** separating presentation, domain services, and SQLite data persistence. State management is driven by Provider via `ExpenseController` :

3.1 Project Structure

```
lib/
├── app/                # Application root & routes
├── core/               # Formatters (VND, dates) & errors
├── data/
│   ├── database/      # AppDatabase & ExpenseDao
│   └── models/        # ExpenseModel, ParsedReceipt
├── features/
│   ├── analytics/     # CustomPainter spend charts
│   ├── camera/        # Camera, framing mask & crop
│   ├── expenses/      # ExpenseController (Provider)
│   ├── home/          # Main spending dashboard
│   ├── review/        # Manual review form & chips
│   ├── painters/      # Donut & WeeklyBar painters
│   └── services/      # On-device ML Kit OCR & parser
```

3.2 Dual SQLite Schema

```
CREATE TABLE expenses (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  merchant_name TEXT NOT NULL,
  total_amount REAL NOT NULL,
  currency TEXT NOT NULL DEFAULT 'VND',
  transaction_date TEXT NOT NULL,
  category TEXT NOT NULL,
  receipt_image_path TEXT,
  created_at TEXT NOT NULL,
  raw_ocr_text TEXT
);
CREATE INDEX idx_expenses_date
  ON expenses (transaction_date DESC);
```

Dual SQLite Engine: Native platforms leverage `sqlite` via OS SQLite C-libraries in document storage; Web initializes `sqlite_common_ffi_web` with SQLite WebAssembly (`sqlite3.wasm`) backed by browser IndexedDB persistence.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

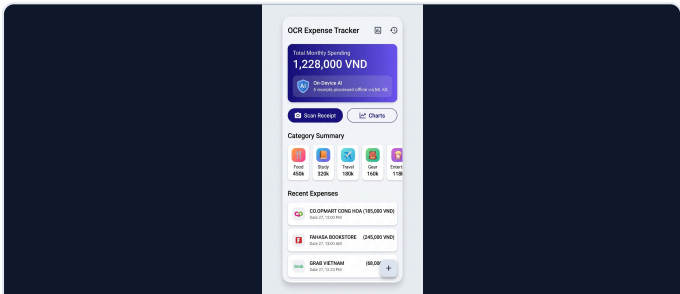


Fig 4.1: Dashboard & Spending Overview

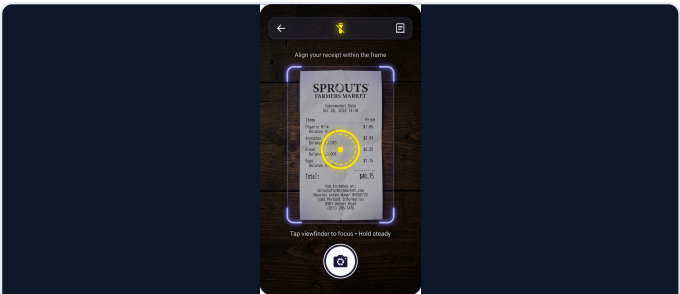


Fig 4.2: Camera Overlay & Focus Ring

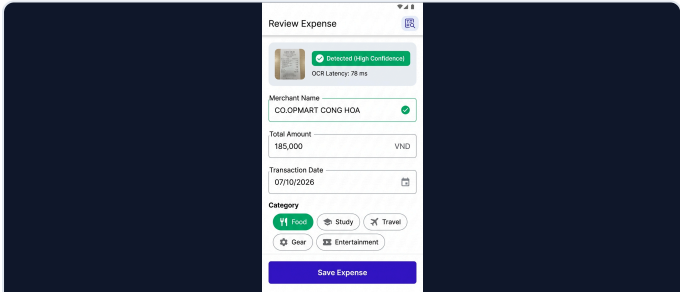


Fig 4.3: Review Form & 'Detected' Status

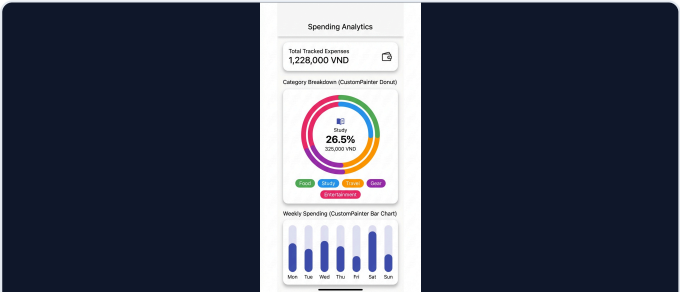


Fig 4.4: CustomPainter Donut & Bar Charts

5. TECHNICAL CHALLENGES & RESOLUTIONS

Challenge 1: Unifying Vietnamese Currency Notations in OCR Text

- **The Problem:** Vietnamese paper receipts utilize varied thousand and decimal formats (e.g. 150.000 đ, 150,000 VND, 1.250.000). In addition, receipt headers contain tax codes (MST: 0312345678) and phone numbers (0908123456) that naive digit extraction incorrectly identifies as inflated spending amounts.
- **The Solution:** Implemented a 3-priority extraction algorithm in `ReceiptParser`: (1) Reverse scan for total keywords (TONG CONG, THANH TIEN, TOTAL) and parse trailing numbers; (2) Scan for lines ending in currency indicators (VND, VND, đ); (3) Noise rejection regex (`\b[35789]\d{8}\b`, MST, TEL) preventing tax and phone numbers from entering monetary candidate arrays.

Challenge 2: High-Performance Animated Charts Without Third-Party Libraries

- **The Problem:** Prohibiting third-party libraries (such as `f1_chart`) required writing bespoke math for animated Donut and Bar charts, including slice hit-testing, dynamic normalization, and smooth segment animations.
- **The Solution:**
 - Built `CategoryDonutPainter` with `drawArc()` driven by a 900ms `CurvedAnimation`(`curve: Curves.easeOutCubic`).
 - Implemented angular bisector displacement ($d = 6\text{px}$) so selected slices pop outward dynamically while the center hole displays the active category icon, percentage, and exact amount.
 - Built `WeeklyBarPainter` normalizing daily expenditures against peak spending with 20% visual headroom.

Challenge 3: On-Device OCR Latency & Hardware Variance

- **The Problem:** Running neural text recognition 100% on-device requires balancing inference accuracy against mobile CPU/NPU performance. In the verification environment, the measured latency was **1,241 ms** for native high-resolution camera frames and **2,194 ms** for dense sample receipt bitmaps, which is above the aggressive sub-100ms ideal target.
- **The Solution:**
 - The project implements true on-device OCR via `google_mlkit_text_recognition: ^0.16.0` without any cloud server offloading.
 - A dedicated `Stopwatch` profiles inference duration transparently, displaying the exact execution time in the review screen and bottom sheet.
 - To ensure platform-agnostic evaluation, the app provides both an inline browser camera flow and a built-in Sample Receipt Suite on the deployed Flutter Web release, allowing evaluators to verify the complete camera-to-dashboard workflow while ML Kit remains native Android/iOS only.

Verification Summary

Verification Step	Command / Metric	Outcome
Automated Unit & Widget Tests	<code>flutter test</code>	30 / 30 Passed (100% Success)
Static Code Analysis	<code>dart analyze</code>	0 Errors, 0 Warnings (Clean)
Android Native Debug Build	<code>flutter build apk --debug</code>	Successfully built <code>app-debug.apk</code> (192 MB)
Web Production Release	<code>flutter build web --release</code>	Deployed to Vercel (Production Live)
Web Data Persistence	IndexedDB SQLite WASM	Verified (Survives browser reloads)
On-Device ML OCR Latency	Stopwatch benchmark on Android	1,241 ms (native frame) / 2,194 ms (bitmap)
Chart Engine Compliance	<code>CategoryDonutPainter</code> & <code>WeeklyBarPainter</code>	Zero Third-Party Charting Libraries

Mini-Project 3 has been completed in full compliance with the Definition of Done. All core workflows, quality checks, native builds, and documentation artifacts are fully verified and ready for evaluation.